Practice 5. Building Binary Search Trees

[CSE2010] Data Structures

Department of Data Science

Practice 3 Solution in C++

```cpp
class Student {
public:
  unsigned long id;
  std::string name;
  Student* next;
  Student(unsigned long i, string n, Student* p = nullptr)
  : id(i), name(n), next(p) {}
};

class Course {
public:
  Student* head;
  Course(): head(nullptr) {}
  ~Course() {
    Student* curr = head;
    Student* next = nullptr;
    while (curr) {
      next = curr->next;
      delete curr;
      curr = next;
    }
  }

  bool isEmpty() {
    return (head == nullptr);
  }

  bool addStudent(int id, string name);
  bool deleteStudent(int id);
  Student* find(int id);
  void write(ofstream& outFile);
};

```

```cpp
bool Course::addStudent(int id, string name) {
  if (isEmpty()) {
    head = new Student(id, name);
    return true;
  }
  // Add a node at the beginning
  if (id < head->id) {
    Student* temp = head;
    head = new Student(id, name);
    head->next = temp;
    return true;
  }
  else {
    Student* prev = nullptr;
    Student* curr = head;
    while (curr) {
      if (curr->id == id) {
        // cerr<<"ADD: duplicate id "<<id<<endl;
        return false;
      }
      if ((prev == nullptr || prev->id < id) && (id < curr->id)) {
        Student* temp = new Student(id, name, curr);
        // Add a node in the middle
        prev->next = temp;
        return true;
      }
      prev = curr;
      curr = curr->next;
    }
    // Add a node at the end
    prev->next = new Student(id, name);
    return true;
  }
}
```

Practice 3 Solution in C++

```
bool Course::deleteStudent(int id) {
  if (isEmpty()) {
    cerr<<"DELETE: no element exists"<<endl;
    return false;
  }
  Student* curr = head;
  Student* prev = nullptr;
  while (curr) {
    if (curr->id == id) {
      if (prev == nullptr) {
        delete curr;
        head = curr->next;
      }
      else {
        prev->next = curr->next;
        delete curr;
      }
      return true;
    }
    else if (curr->id > id) {
      return false;
    }
    prev = curr;
    curr = curr->next;
  }
  return false;
}
```

```
Student* Course::find(int id) {
  if (isEmpty()) {
    cerr<<"FIND: no such element exists"<<endl;
    return nullptr;
  }
  Student* curr = head;
  while (curr) {
    if (curr->id == id) {
      return curr;
    }
    else if (curr->id > id) {
      return nullptr;
    }
    curr = curr->next;
  }
  return nullptr;
}

void Course::write(ofstream& outFile) {
  Student* curr = head;
  while (curr) {
    outFile<<curr->id<<" "<<curr->name<<" ";
    curr = curr->next;
  }
  outFile<<endl;
}
```

Practice 3 Solution in Python

```python
class Student:
  def __init__(self, i, n, p=None):
    self.id = i
    self.name = n
    self.next = p

class Course:
  def __init__(self, l=[]):
    if not l:
      self.head = None
      self.size = 0
    else:
      self.head = Node(l[0])
      self.size = len(l)
      curr = self.head
      for x in l[1:]:
        curr.next = Node(x)
        curr = curr.next

  def __len__(self):
    return self.size

  def isEmpty(self):
    return self.size == 0

  def addStudent(self, newID, newName):
    if self.isEmpty():
      self.head = Student(newID, newName)
      self.size += 1
      return True
    # Add a node at the beginning
    if newID < self.head.id:
      temp = self.head
      self.head = Student(newID, newName, temp)
      self.size += 1
      return True
    else:
      prev = None
      curr = self.head
      while curr:
        if curr.id == newID:
          return False
        if (not prev or prev.id < newID) and (newID < curr.id):
          newNode = Student(newID, newName, curr)
          # Add a node in the middle
          prev.next = newNode
          self.size += 1
          return True
        prev = curr
        curr = curr.next
      # Add a node at the end
      prev.next = Student(newID, newName)
      self.size += 1
      return True

  def deleteStudent(self, queryID):
    if self.isEmpty():
      print("DELETE: no element exists")
      return False
    curr = self.head
    prev = None
    while curr:
      if curr.id == queryID:
        if not prev:
          head = curr.next
        else:
          prev.next = curr.next
        self.size -= 1
        return True
      elif curr.id > queryID:
        return False
      prev = curr
      curr = curr.next
    return False

  def find(self, queryID):
    if self.isEmpty():
      print("FIND: no such element exists")
      return None
    curr = self.head
    while curr:
      if curr.id == queryID:
        return curr
      elif curr.id > queryID:
        return None
      curr = curr.next
    return None

  def print(self):
    curr = self.head
    while curr:
      print(str(curr.id) + " " + str(curr.name) + " ", end="")
      curr = curr.next
    print()

  def write(self, outFile):
    curr = self.head
    while curr:
      outFile.write(str(curr.id) + " " + str(curr.name) + " ")
      curr = curr.next
    outFile.write("\n")
```

TODO

- Implement the following operations of **binary search tree**.
  1. Given a sequence of integers sorted in the ascending order, build a "**balanced**" binary search tree
  2. Find the **minimum** in the tree
  3. Find the **maximum** in the tree
- Write comment for the time complexity of each operation above.

Input of BST

- Each line represents a single operation.
  1. **B<space>([int]+)**
     B<space>[int]<space>[int]<space>…<space>[int]
     - If a sequence of input integers is not sorted in the ascending order, immediately terminate the program and print out an error message.
     - Otherwise, build a balanced binary search tree with these integers as keys, and **write "B" in output file.**
  2. **m**
     - If finding the minimum fails, immediately terminate the program and print out an error message.
     - Otherwise, **write the minimum into output file.**
  3. **M**
     - If finding the maximum fails, immediately terminate the program and print out an error message.
     - Otherwise, **write the maximum into output file.**

Input and Output Examples

- Each line represents to the result of the corresponding line of the input file.
- Input File & Output File

```
B 1 2 3 4 5 6 7 8 9
m
M
```

```
B
1
9
```

```
[hjkim@localhost bst]$ ./practice4 input3.txt output3.txt
                      5

            2                 7

      1         3         6         8

                  4                   9

[hjkim@localhost bst]$ cat input3.txt
B 1 2 3 4 5 6 7 8 9
m
M
[hjkim@localhost bst]$ cat output3.txt
B
1
9
[hjkim@localhost bst]$
```

- Input File & Output File

```
B 1 2 3 5 4 6 7 8 9
m
M
```

```
[hjkim@localhost bst]$ ./practice4 input4.txt output4.txt
BUILD: invalid input
[hjkim@localhost bst]$ cat input4.txt
B 1 2 3 5 4 6 7 8 9
m
M
[hjkim@localhost bst]$ cat output4.txt
[hjkim@localhost bst]$
```

Submission Guideline

- Submission: **source code, input file, output file**
  - Where: Practice 5 submission page in LMS
  - Deadline: **23:59, April. 7th (Sunday)**

Next Practice

- **April 10th (Wednesday)**
  - Traversing a tree with different traversal methods
    - Preorder traversal
    - Inorder traversal
    - Postorder traversal
  - Search, insertion, deletion